NUS National University of Singapore
Department of Statistics and Data Science
Faculty of Science

DSS5201 Data Visualisation

Week 3 Python Basics (Part II)

David CHEW

Semester 2 AY25/26

Last Week: Python Basics Part I

- Python Objects
- Data Types
- Data Structures
- Conditional Executions (`if-elif-else`)

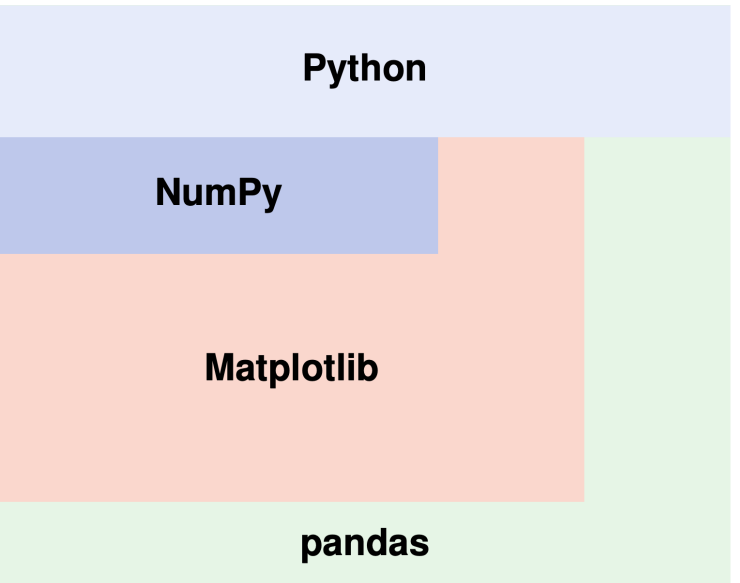
This Week: Python Basics Part II

- Introduction to `NumPy`
- Introduction to `pandas`
- Loops (Repeated Executions)
- Functions
- Writing Readable Code

Core packages

Python has several widely used packages that are essential for data analysis.

- `NumPy` handles numerical computations using arrays and matrices.
- `Matplotlib` builds on `NumPy` and provides a foundation for creating data visualizations.
- `pandas` builds on both `NumPy` and `Matplotlib` and provides data structures like DataFrames for easier data manipulation and data analysis.



NumPy

NumPy: Introduction

- `NumPy` stands for **Numerical Python**.
- A package for scientific computing using arrays and matrices.
- We have already installed the package with `requirements.txt`.
- If you are familiar with MATLAB, you will find this tutorial useful to get started with `NumPy`.

NumPy: Introduction

By convention, we import `NumPy` with the alias `np`.

```
1  import numpy as np
```

- We can create an array from a list with `np.array()`.

```
1  array1 = np.array([1, 20, 43, 14, 5])
2  print(array1)
```

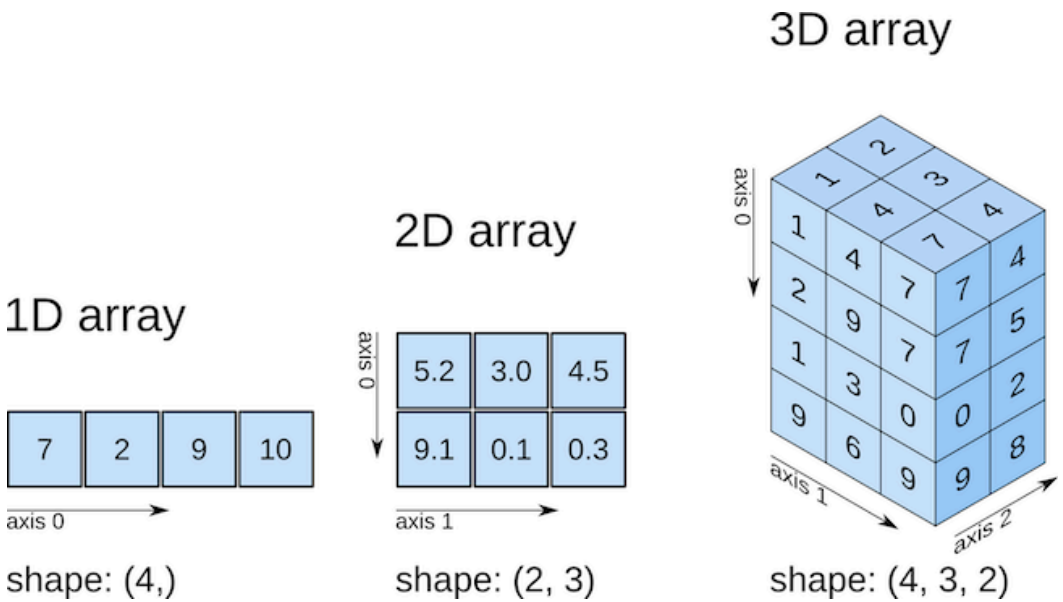
```
[ 1 20 43 14  5]
```

- Vectorized operations work element-wise:

```
1  print(array1 * 2)
```

```
[ 2 40 86 28 10]
```

NumPy: Arrays



Arrays are n-dimensional data structures for fast numerical operations.

- Can hold basic Python data types (e.g., integers, floats).
- Work best with numeric data.
- All elements in an array must be of the same type.

NumPy: Arrays

There are functions to create arrays: `np.arange()`, `np.ones()`, `np.zeros()`, ...

- A sequence of integers from 1 to 5 (end index not included):

```
1  np.arange(1, 6)
```

```
array([1, 2, 3, 4, 5])
```

- A 2×2 array filled with zeros:

```
1  np.zeros((2, 2))
```

```
array([[0., 0.],
       [0., 0.]])
```

NumPy: Simulations

`NumPy` has built-in random number generators.

- Generate 5 random numbers from a uniform distribution over $[0, 1]$:
  - To make the numbers reproducible, we set the random seed with `np.random.seed()`.

```
1  np.random.seed(5201)
2  np.random.rand(5)
```

```
array([0.50452554, 0.09645924, 0.71865443, 0.32890398, 0.88347242])
```

- Draw 5 random numbers from a normal distribution:

```
1  np.random.normal(loc = 0, scale = 1, size = 5)
```

```
array([-0.13981544, -0.70121702,  0.0400001 ,  1.73731715, -0.85599173])
```

NumPy: Simulations

- Randomly select element(s) from a given list:

```
1  np.random.choice([1, 3, 5, 9], size = 2, replace = True)
```

```
array([5, 1])
```

- To simulate rolling two dice:

```
1  np.random.choice(range(1, 7), size = 2, replace = True)
```

```
array([6, 5])
```

NumPy: Accessing Elements

We can slice arrays just like Python lists.

- The syntax is `x[start:stop:step]`

```
array2 = np.arange(1, 6)
array2[1:4]
```

```
array([2, 3, 4])
```

- We can also omit the start or stop:

```
array2[:4]     # First four elements
```

```
array([1, 2, 3, 4])
```

NumPy: Sorting

We can sort arrays of numeric, strings, or booleans. There are two main ways:

- `x.sort()` sorts the original array inplace. It modifies and overwrites the original array.

```
array1 = np.array([1, 20, 43, 14, 5])
array1.sort()
print(f'The array is sorted as {array1}.')
```

```
The array is sorted as [ 1  5 14 20 43].
```

- `np.sort(x)` returns a new array, without changing the original one.

```
array1 = np.array([1, 20, 43, 14, 5])
sorted1 = np.sort(array1)
print(f'The original array is {array1}, the new one is {sorted1}.')
```

```
The original array is [ 1 20 43 14  5], the new one is [ 1  5 14 20 43].
```

NumPy: Sorting

- For arrays of strings, elements are sorted alphabetically.

```
array3 = np.array(['pigeon', 'cat', 'lion', 'dog'])
array3.sort()
print(array3)
```

```
['cat' 'dog' 'lion' 'pigeon']
```

- For arrays of booleans, `False` is treated as 0 and `True` as 1. Sorting them will place `False` first.

```
array4 = np.array([True, False, True])
array4.sort()
print(array4)
```

```
[False  True  True]
```

NumPy: Sorting

To reverse the orders, we need an extra step:

- `[::-1]` accesses elements backwards, essentially a reverse copy of the array.

```
array3 = np.array(['pigeon', 'cat', 'lion', 'dog'])
array3.sort()
array3 = array3[::-1]
print(array3)
```

```
['pigeon' 'lion' 'dog' 'cat']
```

NumPy: Summary Statistics

To summarize arrays, we can use `np.mean()`, `np.median()`, `np.std()`, `np.max()`, ...

```
1  array1 = np.array([1, 20, 43, 14, 5])
2  np.mean(array1)
```

```
np.float64(16.6)
```

```
1  np.median(array1)
```

```
np.float64(14.0)
```

```
1  np.std(array1)
```

```
np.float64(14.786480311419618)
```

NumPy: Missing Values

In `NumPy`, missing values are represented as `np.nan` (not a number).

- A special float that indicates missing data.
- It behaves like numeric, but can affect calculations if not handled properly.

```
1  array5 = np.array([1, 20, 43, 14, 5, np.nan])
2  array5
```

```
array([ 1., 20., 43., 14.,  5., nan])
```

- Taking the mean of an array with missing value will return `nan`.

```
1  np.mean(array5)
```

```
np.float64(nan)
```

NumPy: Handling Missing Values

- To ignore missing values in such computations, use functions like `np.nanmean()`:

```
1  np.nanmean(array5)
```

```
np.float64(16.6)
```

```
1  np.nanmedian(array5)
```

```
np.float64(14.0)
```

NumPy: Handling Missing Values

- Given an array, we can use `np.isnan()` to check whether it contains missing values.
- Returns a boolean array that indicates which elements are missing.

```
1  np.isnan(array5)
```

```
array([False, False, False, False, False,  True])
```

- To remove missing values:

```
1  array5_clean = array5[~np.isnan(array5)]
2  array5_clean
```

```
array([ 1., 20., 43., 14.,  5.])
```

NumPy: Array Attributes & Methods

Here are some of the common attributes and methods of a `NumPy` array.

	Type	Description
`.shape`	Attribute	Returns the dimensions of an array
`.T`	Attribute	Transposes the view of an array
`.size`	Attribute	Returns the total number of elements in an array
`.sort()`	Method	Sorts elements in ascending order by default
`.mean()`	Method	Computes column or row-wise mean
`.sum()`	Method	Computes column or row-wise sum
`.argmax()`	Method	Returns the index of the maximum value.
`.reshape()`	Method	Changes the shape of the array.

NumPy: Attributes and Methods

In Python,

- **Attributes** describe the properties of an object. Use without parentheses.

```
1  array1 = np.array([1, 20, 43, 14, 5])
2  array1.size
```

```
5
```

- **Methods** are actions we can perform on an object. Require parentheses.

```
1  array1.argmax()
```

```
np.int64(2)
```

NumPy: Methods and Functions

We can perform actions with **methods** or **functions**.

Here are some examples in `NumPy`:

	Type	Inplace?	Output
`x.sort()`	Method	Yes	`None`
`np.sort(x)`	Function	No	A sorted, new copy of `x`
`x.mean()`	Method	No	Mean of `x`
`np.mean(x)`	Function	No	Mean of `x`

pandas

pandas: Introduction

The next important library we shall work with is `pandas`.

- Builds on `NumPy`'s foundation and provides powerful tools for data manipulation and analysis.
- Two main data structures: `Series` and `DataFrame`.
- By convention, we import `pandas` with the alias `pd`.

```
1 import pandas as pd
```

pandas: Series

- A **series** is a one-dimensional array-like object in pandas.
- Similar to a NumPy array, but with row labels (indices) for each element.
- By default, the index starts at 0.

```
1 series1 = pd.Series([1, 2, 3, 4, 5])
2 series1
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

pandas: Series

We can assign a custom index with the `index` argument.

```
1  series2 = pd.Series([200, 350, 201],
2                      index = ['Jan', 'Feb', 'Mar'])
3  series2
```

```
Jan    200
Feb    350
Mar    201
dtype: int64
```

pandas: Series

We can also assign a custom index with a dictionary.

- The `key` become the index and values become the data.

```
1 series2 = pd.Series({'Jan': 200, 'Feb': 350, 'Mar': 201})
2 series2
```

```
Jan    200
Feb    350
Mar    201
dtype: int64
```

pandas: Accessing Elements

There are two main ways to access elements in a series:

- By position with a pair of square brackets `[]`:

```
1  series2[0:2]
```

```
Jan    200
Feb    350
dtype: int64
```

- By labels, similar to how we access values in dictionaries:

```
1  series2['Feb']
```

```
np.int64(350)
```

pandas Series vs NumPy Arrays

`pandas` series are more flexible than `NumPy` arrays.

- Custom index makes data interpretable and easier to work with.
- Values are aligned by index before arithmetic operations:

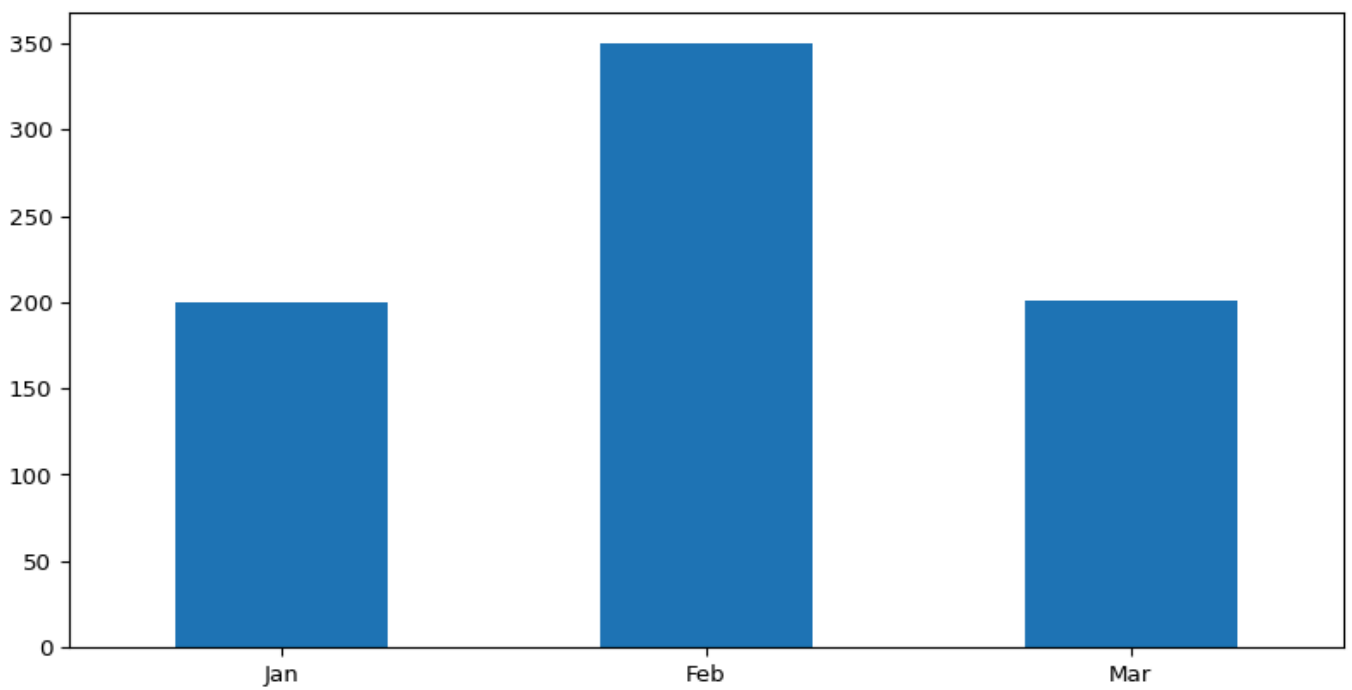
```
sales2025 = pd.Series([300, 480], index = ['Jan', 'Feb'])
sales2024 = pd.Series([470, 350], index = ['Feb', 'Jan'])
sales2025 - sales2024
```

```
Feb    10
Jan   -50
dtype: int64
```

pandas Series vs NumPy Arrays

When we visualize a series, its index becomes the x-axis automatically.

```
1 import matplotlib.pyplot as plt
2 series2.plot(kind = 'bar')
3 plt.xticks(rotation = 0)
4 plt.show()
```



Code Explained

```
import matplotlib.pyplot as plt
series2.plot(kind = 'bar')
plt.xticks(rotation = 0)
plt.show()
```

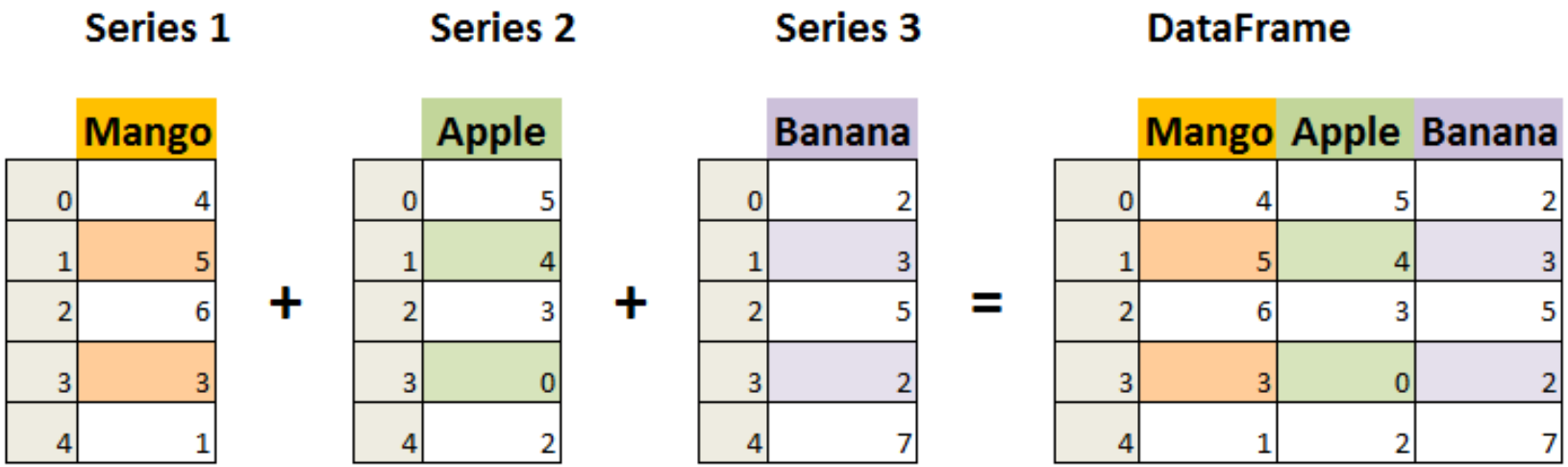
`pandas`' built-in plotting method `.plot()` builds on `matplotlib`.

- `kind = 'bar'` creates a vertical bar chart, with index labels displayed on the x-axis automatically.

The last two lines are `matplotlib` functions:

- `plt.xticks()` controls the x-axis labels.
- `plt.show()` displays the plot.

pandas DataFrames



DataFrames are multiple **series** combined side by side.

- Can be created using `pd.DataFrame()`.
- Index and column labels starts from `0` by default.

pandas: DataFrames vs Series

- A **series** is a single column of data with row index.

```
1  sales2025 = pd.Series([300, 480], index = ['Jan', 'Feb'])
2  sales2024 = pd.Series([470, 350], index = ['Feb', 'Jan'])
```

- A **DataFrame** is a table, with **series** in columns that share the same row index. Each column has a name.

```
1  sales = pd.DataFrame({'2025': sales2025, '2024': sales2024})
2  sales
```

	2025	2024
Feb	480	470
Jan	300	350

pandas: DataFrames

We can create a **DataFrame** from a list.

- Use `index` and `columns` to name the rows and columns.

```
df = pd.DataFrame([['Tommy', 'Python', 5],
                   ['Miley', 'R', 4],
                   ['Tiffany', 'R', 7]],
                   columns = ['Name', 'Language', 'Hours'])
df
```

	Name	Language	Hours
0	Tommy	Python	5
1	Miley	R	4
2	Tiffany	R	7

pandas: DataFrames

Another common way is to use a dictionary:

- Each key becomes a column name and each value is a list of entries in the column.
- Generally easier to read and maintain.

```
df = pd.DataFrame({'Name': ['Tommy', 'Miley', 'Tiffany'],
                   'Language': ['Python', 'R', 'R'],
                   'Hours studied': [5, 4, 7]})
df
```

	Name	Language	Hours studied
0	Tommy	Python	5
1	Miley	R	4
2	Tiffany	R	7

pandas: Subsetting a Column

In data analysis, we often need to subset a data frame.

- There are two main ways to extract a single column. The resulting object is a `pandas` series.
- We use **dot notation** if the column name is a valid Python identifier.
  - No spaces, no special characters, does not conflict with names of built-in methods (e.g., `count`, `sum`).

```
1 df.Name
```

```
0       Tommy
1       Miley
2    Tiffany
Name: Name, dtype: object
```

pandas: Subsetting a Column

If the column name is **not a valid Python identifier**, we cannot use the dot notation.

```
df.Hours studied        # SyntaxError
```

We should use the **bracket notation** instead:

```
df['Hours studied']
```

```
0    5
1    4
2    7
Name: Hours studied, dtype: int64
```

pandas: Subsetting Rows

Another common task is to filter rows based on certain conditions.

We use boolean expressions inside brackets:

```
1  df[df['Language'] == 'R']
```

	Name	Language	Hours studied
1	Miley	R	4
2	Tiffany	R	7

```
1  df[df['Hours studied'] > 5]
```

	Name	Language	Hours studied
2	Tiffany	R	7

pandas: Subsetting Rows

We can combine conditions with & (and), | (or), and ~ (not).

Always wrap conditions in parentheses when combining them:

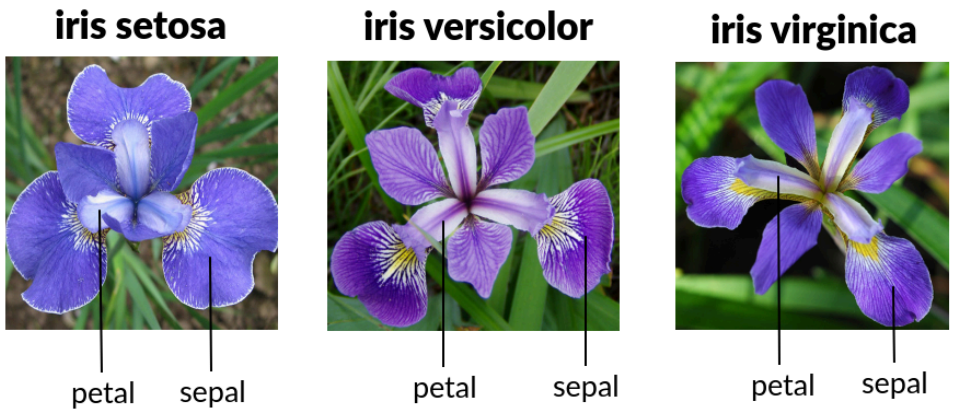
```
df[(df['Language'] == 'R') & (df['Hours studied'] >= 4)]
```

	Name	Language	Hours studied
1	Miley	R	4
2	Tiffany	R	7

```
df[~(df['Language'] == 'R')]
```

	Name	Language	Hours studied
0	Tommy	Python	5

Iris Data Set



A classic data set on iris plants through the `pydataset` package.

- Measurements of three iris species.
- Each row corresponds to one flower with features like sepal and petal length/width.

```
from pydataset import data
iris = data('iris')
iris.head(1)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa

Iris Data Set: Accessing Column Names

- To access column names of a DataFrame:

```
1  iris.columns
```

```
Index(['Sepal.Length', 'Sepal.Width', 'Petal.Length', 'Petal.Width',
       'Species'],
      dtype='object')
```

- How to select the column `Sepal.Length`?

Iris Data Set: Renaming Columns

We can rename the columns to follow the “snake_case”.

- One way is to type out the new column names by their positions.
- Easy to understand, but not scalable (try renaming 10+ columns by hand).

```
col_names = ['sepal_length', 'sepal_width',
             'petal_length', 'petal_width', 'species']
iris.columns = col_names
iris.head(1)
```

	sepal_length	sepal_width	petal_length	petal_width	species
1	5.1	3.5	1.4	0.2	setosa

Iris Data Set: Renaming Columns Programmatically

Let's automate and standardize the renaming process:

- We first create an empty list to store the new column names.
- `.lower()` is a method that converts the column names to lowercase.
- `.replace('.', '_')` replaces each dot symbol with an underscore.
- `.append()` adds the cleaned names to the list.

```
iris = data('iris')
col_names = []
for x in iris.columns:
    cleaned_name = x.lower().replace('.', '_')
    col_names.append(cleaned_name)
iris.columns = col_names
iris.head(1)
```

	sepal_length	sepal_width	petal_length	petal_width	species
1	5.1	3.5	1.4	0.2	setosa

Iris Data Set: Renaming Columns with List Comprehension

We can rewrite the same logic in a more compact way with list comprehension.

- Iterates over `iris.columns` and applies the string operations:

```
iris = data('iris')
iris.columns=[x.lower().replace('.', '_') for x in iris.columns]
iris.head(1)
```

	sepal_length	sepal_width	petal_length	petal_width	species
1	5.1	3.5	1.4	0.2	setosa

pandas: High-level Summaries

Here are 3 helpful attributes and methods for getting **high-level summaries** for **DataFrames**.

- `.shape` gives the number of rows and columns.
- `.unique()` returns distinct values in a column (useful for strings and categories).
- `.describe()` provides summary statistics of numeric columns by default.

```
1  iris.shape
```

```
(150, 5)
```

```
1  iris['species'].unique()
```

```
array(['setosa', 'versicolor', 'virginica'], dtype=object)
```

pandas: Summarizing a DataFrame

`.dtypes` shows data types of each column.

- Recall that `float` represents numeric values with decimals.
- In `pandas`, `object` typically means strings.

```
1 iris.dtypes
```

```
sepal_length    float64
sepal_width     float64
petal_length    float64
petal_width     float64
species          object
dtype: object
```

pandas: Summarizing a DataFrame

By default, `.describe()` returns summaries for numeric columns.

```
1 iris.describe()
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

pandas: Summarizing a DataFrame

With `include = 'all'`, we can include summaries for both numeric and string columns.

```
1  iris.describe(include = 'all')
```

	sepal_length	sepal_width	petal_length	petal_width	species
count	150.000000	150.000000	150.000000	150.000000	150
unique	NaN	NaN	NaN	NaN	3
top	NaN	NaN	NaN	NaN	setosa
freq	NaN	NaN	NaN	NaN	50
mean	5.843333	3.057333	3.758000	1.199333	NaN
std	0.828066	0.435866	1.765298	0.762238	NaN
min	4.300000	2.000000	1.000000	0.100000	NaN
25%	5.100000	2.800000	1.600000	0.300000	NaN
50%	5.800000	3.000000	4.350000	1.300000	NaN
75%	6.400000	3.300000	5.100000	1.800000	NaN
max	7.900000	4.400000	6.900000	2.500000	NaN

pandas: Summarizing a DataFrame

To summarize a non-numeric column, we can use `.value_counts()` to display how often each value appears.

```
1 iris['species'].value_counts()
```

```
species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```

pandas: Summarizing a DataFrame

`.min()` returns the minimum value found in each column. For strings, it returns the first value in alphabetical order.

```
1  iris.min()
```

```
sepal_length        4.3
sepal_width         2.0
petal_length        1.0
petal_width         0.1
species          setosa
dtype: object
```

To extract the minimum of a particular column:

```
1  iris['sepal_length'].min()
```

```
np.float64(4.3)
```

pandas: Sorting Rows by Index

We can sort the data frame based on the row index with `.sort_index()`.

By default, it does not change the original data frame, but we can modify the default behavior with `inplace = True`.

```
1 iris.sort_index(ascending = False).head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
150	5.9	3.0	5.1	1.8	virginica
149	6.2	3.4	5.4	2.3	virginica
148	6.5	3.0	5.2	2.0	virginica
147	6.3	2.5	5.0	1.9	virginica
146	6.7	3.0	5.2	2.3	virginica

pandas: Sorting Rows by Value

`.sort_values()` sorts the data frame based on values in specific column(s).

```
1 iris.sort_values(by = 'sepal_length', ascending = True).head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
14	4.3	3.0	1.1	0.1	setosa
43	4.4	3.2	1.3	0.2	setosa
39	4.4	3.0	1.3	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
42	4.5	2.3	1.3	0.3	setosa

pandas: Sorting Rows by Value

To control how ties are broken, we can sort by multiple columns. More predictable and reproducible.

```
iris.sort_values(by = ['sepal_length', 'sepal_width'],
                 ascending = [True, True]).head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
14	4.3	3.0	1.1	0.1	setosa
9	4.4	2.9	1.4	0.2	setosa
39	4.4	3.0	1.3	0.2	setosa
43	4.4	3.2	1.3	0.2	setosa
42	4.5	2.3	1.3	0.3	setosa

pandas: Adding Columns

We can create a column directly with `[]`.

```
1 iris['sepal_length_mm'] = iris['sepal_length'] * 10
2 iris.head(2)
```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_length_mm
1	5.1	3.5	1.4	0.2	setosa	51.0
2	4.9	3.0	1.4	0.2	setosa	49.0

pandas: Dropping Columns

Use `.drop()` to remove columns.

```
iris = iris.drop(columns = ['sepal_length_mm'])
iris.head(2)
```

	sepal_length	sepal_width	petal_length	petal_width	species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa

Alternatively, add `inplace = True` to modify the data frame directly:

```
iris.drop(columns = ['sepal_length_mm'], inplace = True)
```

pandas: Adding/Dropping Rows

We can **add or drop rows** of a data frame in two ways:

- Use `.concat()` to add new rows.
- Use `.drop()` to remove rows.

We would not often be doing this manually, but it is good to know how to do it when needed.

pandas: Adding Rows

Create a new row as a data frame, then add it to `iris` with `.concat()`.

```
another_row = pd.DataFrame([[4.3, 2.0, 1.0, 0.1, 'new class']],
                           columns = iris.columns, index = [151])
iris = pd.concat([iris, another_row])
iris.tail()
```

	sepal_length	sepal_width	petal_length	petal_width	species
147	6.3	2.5	5.0	1.9	virginica
148	6.5	3.0	5.2	2.0	virginica
149	6.2	3.4	5.4	2.3	virginica
150	5.9	3.0	5.1	1.8	virginica
151	4.3	2.0	1.0	0.1	new class

pandas: Dropping Rows

We can remove a row with `.drop()` by index:

```
1  iris = iris.drop(index = [149, 151])
2  iris.tail()
```

	sepal_length	sepal_width	petal_length	petal_width	species
145	6.7	3.3	5.7	2.5	virginica
146	6.7	3.0	5.2	2.3	virginica
147	6.3	2.5	5.0	1.9	virginica
148	6.5	3.0	5.2	2.0	virginica
150	5.9	3.0	5.1	1.8	virginica

pandas: DataFrame Methods

Here are some of the `pandas` attributes and methods we covered:

	Type	Description
`.shape`	Attribute	Returns the number of rows and columns of a data frame
`.dtypes`	Attribute	Prints the data types of columns
`.columns`	Attribute	Lists all column names
`.describe()`	Method	Provides summary statistics of numeric columns
`.head()` `.tail()`	Method	Shows the first/ last few rows of the data frame
`.min()` `.max()`	Method	Shows the minimum/ maximum of the data frame
`.value_counts()`	Method	Returns counts of unique values in a categorical column
`.sort_index()`	Method	Sorts the data frame by row index
`.sort_values()`	Method	Sorts the data frame by one or more columns
`.concat()` `.drop()`	Method	Adds/ removes rows or columns from the data frame

Loops

Loops aka Repeated Executions

- Many Python objects (e.g., lists, tuples, dictionaries) are iterable.
- When we want to perform a small task multiple times, we can use a *for loop*.
- For example, we used a *for loop* to clean up column names in `iris`:

```
iris = data('iris')
col_names = []
for x in iris.columns:
    cleaned_name = x.lower().replace('.', '_')
    col_names.append(cleaned_name)
iris.columns = col_names
iris.head(2)
```

Example: For Loop

- Recall that code blocks in Python are defined by **indentation**.
- Lines with the same indentation level belong to the same code block.

```
year = [2024, 2025, 2026, 2027, 2028]
for i in year:
    print(f'The year is {i}.')
```

```
The year is 2024.
The year is 2025.
The year is 2026.
The year is 2027.
The year is 2028.
```

Example: For Loop

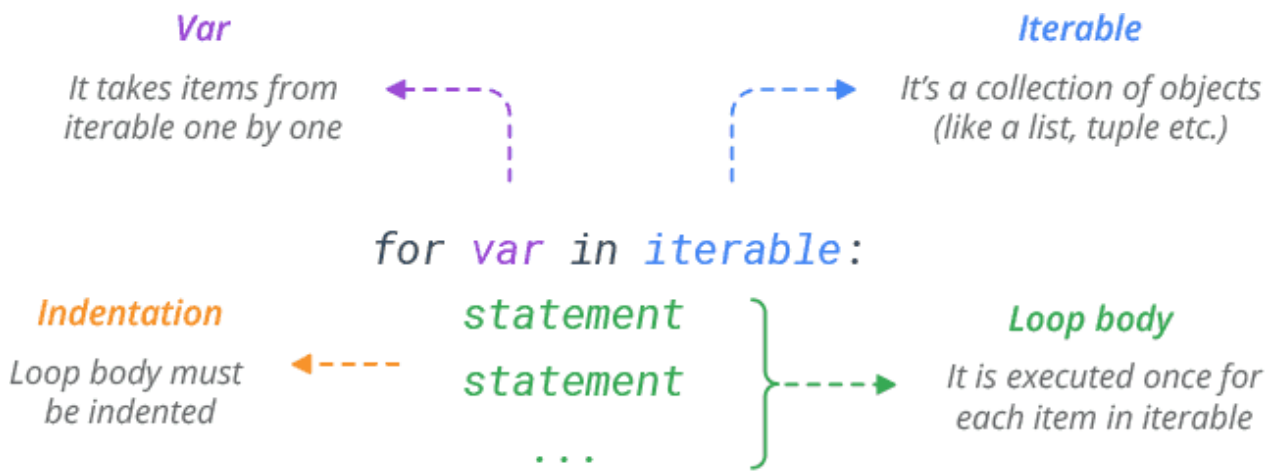
Here is a *for loop* with conditional logic:

```
for i in year:
    if i == 2026:
        print(f'{i} is the current year.')
    else:
        print(f'{i} is not the current year.')
```

```
2024 is not the current year.

2025 is not the current year.

2026 is the current year.

2027 is not the current year.

2028 is not the current year.
```

For Loop

In a basic *for loop*, we specify what has to be done and for how many times.



Example: While Loop

A *while loop* is used when we want to perform a task indefinitely as long as a condition is `True`. The flow of execution is:

1. Test the condition.
2. If `False`, exit the loop.
3. If `True`, run the code block, then go back to Step 1.

```
number = 1
while number <= 2:
    print(number)
    number += 1
```

```
1
2
```

Example: While Loop

The following statements are equivalent:

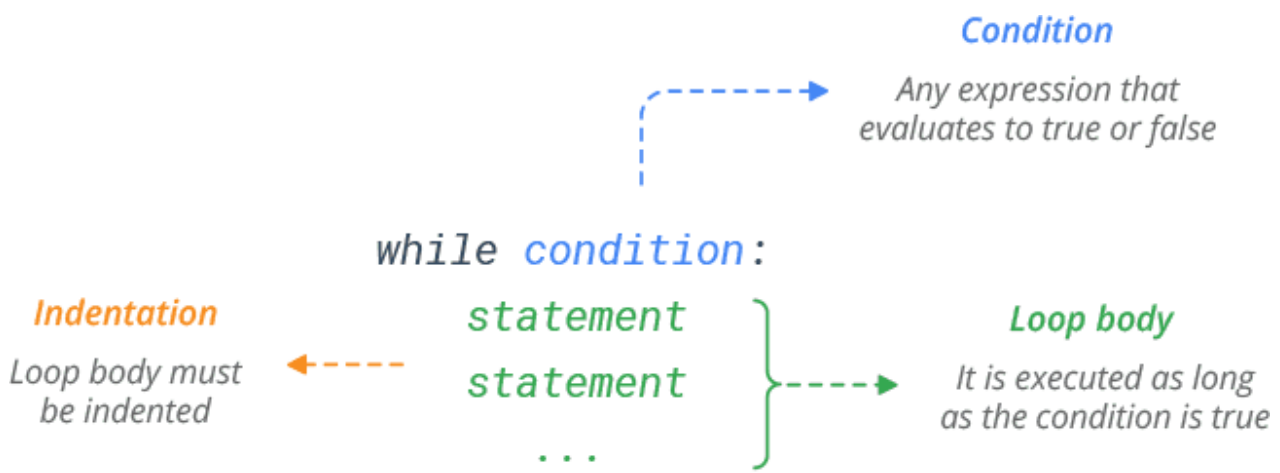
```
1  # 1
2  number += 1
3  # 2
4  number = number + 1
```

Be careful with *while* loops:

- If the condition never becomes `False`, the loop will run forever.
- Always ensure something in the loop updates variables so the condition can eventually fail.

While Loop

A *while* loop is used when we want to perform a task indefinitely, as long as a condition is `True`.



Loops: Use with Caution

	Best Use	Common Pitfalls
`for`	Ideal when the number of iterations is known	Less readable
`while`	Ideal when we want to be flexible about the stopping condition	Risk of infinite loops

If not written properly, loops can get **very inefficient (slow)** when applied to large data sets.

- Still useful, but not always the best tool.
- As you learn more, you will find that **vectorization** is preferred over loops: They are more efficient and produce cleaner and more readable code.

🎲 Dice Roll Simulation: Single roll

We've learned to use NumPy to generate random numbers.

Let's simulate a dice roll game.

```
np.random.seed(5201)
one_roll = np.random.choice([1, 2, 3, 4, 5, 6])
print(f'We rolled a {one_roll}.')
```

```
We rolled a 2.
```

🎲 Dice Roll Simulation: 1000 rolls with *for* loop

Next, simulate a game of 1000 rolls. We use a *for* loop here.

```
np.random.seed(5201)
multiple_rolls = []
for i in range(1000):
    one_roll = np.random.choice([1, 2, 3, 4, 5, 6])
    multiple_rolls.append(one_roll)
```

 Dice Roll Simulation: 1000 rolls with *while* loop

How to simulate the game with a *while* loop?

```
np.random.seed(5201)
multiple_rolls = []
count = 0
while count < 1000:
    one_roll = np.random.choice([1, 2, 3, 4, 5, 6])
    multiple_rolls.append(one_roll)
    count += 1
```

- This is equivalent to the *for* loop.
- *While* loops are generally more flexible and particularly useful when the stopping condition is dynamic (e.g., stop as soon as we roll a six).

🎲 Dice Roll Simulation: Summarizing Results

- To summarize the results, we first convert the results into a `series`.
- `.value_counts()` returns a new `series` with dice roll results as row index and frequencies as values.

```
1 results = pd.Series(multiple_rolls)
2 results.value_counts()
```

```
6    176
1    173
5    173
2    165
4    162
3    151
Name: count, dtype: int64
```

🎲🎲 Dice Roll Simulation: Sum of Two Dice

Let's roll two dice and record the sum in each round. Repeat this 1000 times.

```
np.random.seed(5201)
sum_rolls = []
for i in range(1000):
    two_rolls = np.random.choice([1, 2, 3, 4, 5, 6], size = 2)
    total = two_rolls.sum()
    sum_rolls.append(total)
```

🎲🎲 Dice Roll Simulation: Sum of Two Dice

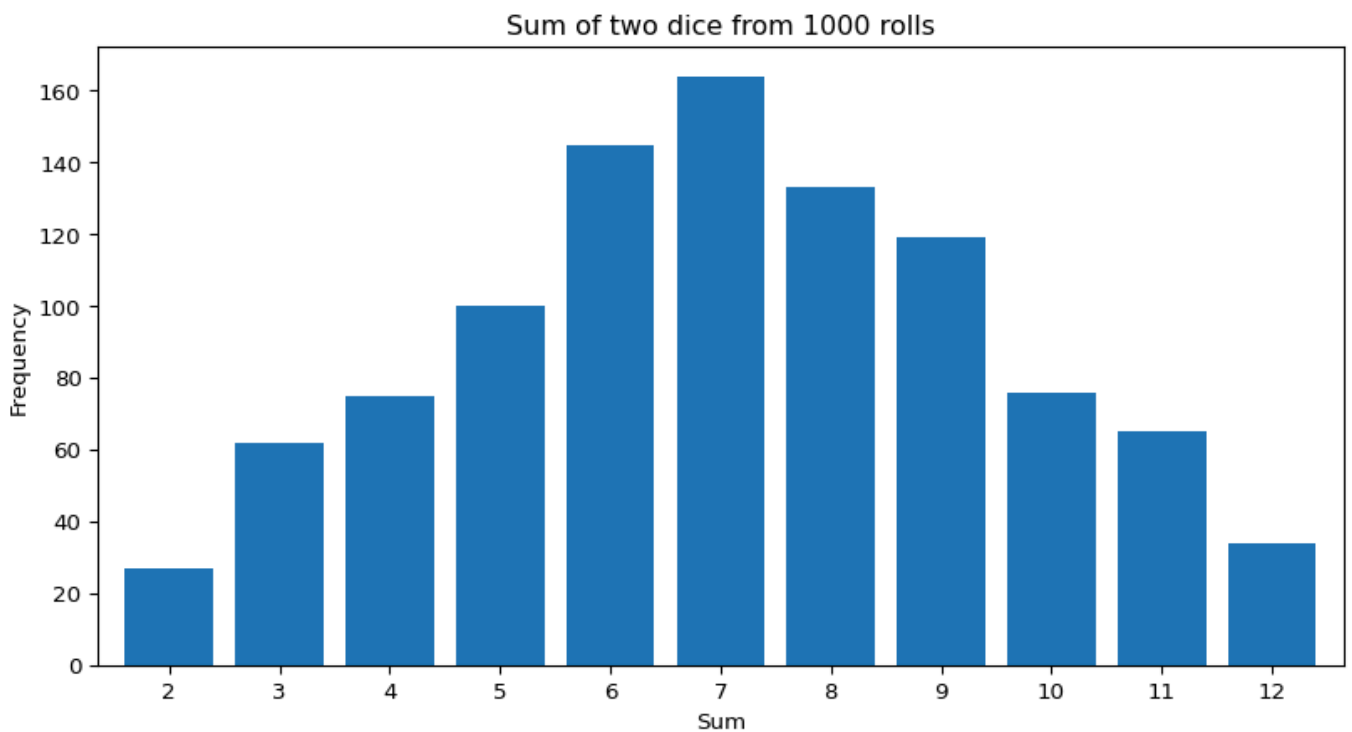
- `sum_rolls` has all 1000 sums.
- We use `.value_counts()` to count the frequency of each outcome.
- Then `.sort_index()` to arrange the results in order from 2 to 12.

```
1 counts = pd.Series(sum_rolls).value_counts().sort_index()
2 counts.head()
```

```
2     27
3     62
4     75
5    100
6    145
Name: count, dtype: int64
```

🎲🎲 Dice Roll Simulation: Sum of Two Dice

```
counts.plot(kind = 'bar',
            title = 'Sum of two dice from 1000 rolls',
            xlabel = 'Sum', ylabel = 'Frequency', width = 0.8)
plt.xticks(rotation = 0)
plt.show()
```



Functions

Python Functions

We have already seen and used functions in Python.

```
help(np.arange)
```

✓ 0.0s

```
Help on built-in function arange in module numpy:

arange(...)
    arange([start,] stop[, step,], dtype=None, *, device=None, like=None)

    Return evenly spaced values within a given interval.

    ``arange`` can be called with a varying number of positional arguments:

    * ``arange(stop)``: Values are generated within the half-open interval
      ``[0, stop)`` (in other words, the interval including `start` but
      excluding `stop`).
    * ``arange(start, stop)``: Values are generated within the half-open
      interval ``[start, stop)``.
    * ``arange(start, stop, step)`` Values are generated within the half-open
      interval ``[start, stop)``, with spacing between values given by
      ``step``.
```

required

optional

Common patterns

Python Functions

Two types of arguments for each Python function:

- Required arguments: We must provide a value when we call the function.
- Optional arguments:
  - shown inside square brackets.
  - If not provided, the function will use a pre-defined default value (online documentation).

For `np.arange()`, only `stop` is required:

```
1 np.arange(5)
```

```
array([0, 1, 2, 3, 4])
```

User-defined Functions

At some point, we will need to write our own function.

- A re-usable piece of code that can take inputs (arguments) and return desired outputs.

We will need to decide:

1. What argument(s) should it take.
2. Whether to assign default values to any of the argument(s).
3. What output should it return.

Typical approach: Write a sequence of expressions that work, then wrap them in a function.

User defined Functions

The syntax for creating a new function in Python is as follows:

```
def function_name(arguments):
    # Python statements
    ...
    return object
```

We shall practice writing a function that calculate the final price of a grocery store item, after taking into account the prevailing GST.

Example

```
def calculate_total(price, tax_rate):
    final_price = price * (1 + tax_rate/100)
    message = f'The final price is {final_price:,.2f}.'
    return message
```

- The function takes two required arguments: `price` and `tax_rate`.
- It does not have default values. We must provide the values when calling the function.

```
calculate_total(price = 1000, tax_rate = 9)
```

```
'The final price is 1,090.00.'
```

Example

- If we do not supply values for the required arguments, we will get a `TypeError`.
- When we declare an argument with a default value, the argument becomes optional.
- The default value will be applied if the argument is left empty.

```
1 def calculate_total_tax(price, tax_rate = 9):
2     final_price = price * (1 + tax_rate/100)
3     message = f'The final price is {final_price:,.2f}.'
4     return message
5
6 calculate_total_tax(price = 1000)
```

```
'The final price is 1,090.00.'
```

Anonymous (`lambda`) Functions

Anonymous functions provide a compact way of defining a function without assigning it a name.

Consider the following simple function:

```
def extract_id(x):
    return x[0:8]
extract_id('e0123456@nus.edu.sg')
```

```
'e0123456'
```

Example

We can define an equivalent anonymous function with the `lambda` keyword:

```
1 extract_id_lambda = lambda x: x[0:8]
2 extract_id_lambda('e0123456@nus.edu.sg')
```

```
'e0123456'
```

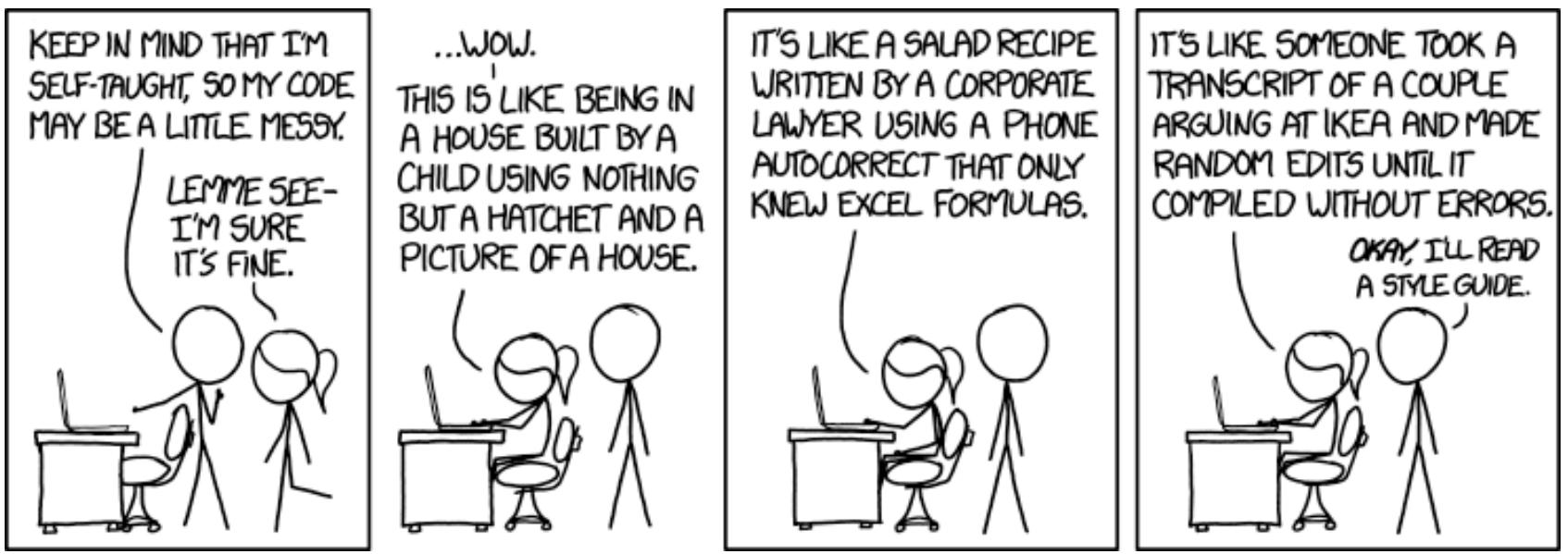
- Use lambda only for very short and obvious one-off tasks.
- Otherwise your code will be unreadable and very difficult to debug and maintain.

Writing Readable Code

The PEP 8 Python Style Guide

The Python Enhancement Proposal (PEP 8) is the official style guide for Python.

- The goal is to write clean code with good style.
- **Readability:** Code should be easy to understand at a glance.



The Zen of Python

```
1  import this
```

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

Indentation Requirements

Indentation

- PEP 8 advise us to use 4 spaces per indentation level.
- In VS Code, we can insert 4 spaces with the `Tab` key by default.
- Improper indentation can lead to `SyntaxError`'s.

```
1 weather = 'Rainy'
2 wind = 'Windy'
3 if (weather == 'Rainy') and (wind == 'Windy'):
4     print('Just stay at home')
```

```
Just stay at home
```

Readable Code

Single space between items

- Most code have a mix of variables and operators such as `=`, `==`, `>=`, etc.
- Use a single space to separate both sides of the operator for readability.

```
age = 25
if age >= 20:
    print(f'{age - 2} years')
```

```
23 years
```

- A comma or colon has one space after it, but no spaces before it.
- The left and right brackets do not have spaces separating them from others.

Readable Code

Break up long line of code

- If a line is long, we can break it up and indent the later lines:

```
sg_towns = ['Clementi', 'Ang Mo Kio', 'Bishan', 'Bukut Timah',
            'Jurong East', 'Pioneer', 'Central']
```

Use a consistent quote mark

- Python strings can be written in single or double quotes.
- PEP 8 requires a program to pick one code style and use it consistently.

Readable Code

Break up long line of code

- When a function has many arguments, we can break the line after each comma and align the arguments vertically.
- Everything inside parentheses `()` is treated as one code chunk. Python will read them together until the closing bracket.
- Both styles below are acceptable:

```
# 1 Aligned after commas
counts.plot(kind = 'bar',
            title = 'Sum of two dice from 1000 rolls',
            xlabel = 'Sum', ylabel = 'Frequency', width = 0.8)

# 2 One (or more) argument per line
counts.plot(
    kind = 'bar',
    title = 'Sum of two dice from 1000 rolls',
    xlabel = 'Sum', ylabel = 'Frequency',
    width = 0.8
)
```

Readable Code

Break up long line of code

- When we chain multiple methods, we can wrap them all in one line (#1).
- ... or split them in multiple lines inside parentheses after each method (#2).
- Both styles below are acceptable:

```
# 1 All in one line
counts = pd.Series(sum_rolls).value_counts().sort_index()

# 2 Line break inside parentheses after each method
counts = (
    pd.Series(sum_rolls)
    .value_counts()
    .sort_index()
)
```

Readable Code

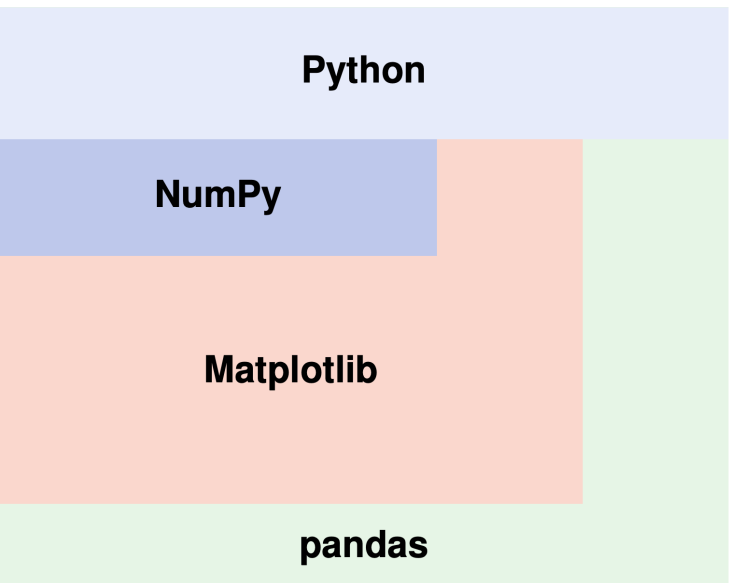
Other important rules for readability:

- Import all necessary packages at the start of your script.
- Keep your lines of code to a maximum of 79 characters.
- Use comments to explain the code when necessary.
  - Single-line comments start with the `#` symbol.
  - In-line comments can be placed at the end of a line, also starts with `#`.
  - Multi-line comments with triple quotes start with `'''` and end with another `'''`.

Summary

Python Basics (Part II)

- Introduction to `NumPy`
- Introduction to `pandas`
- Loops (Repeated Execution)
- Functions
- Writing Readable Code



Next class:

- Data import and exploration with `pandas` and `Matplotlib`.